Academic Year 2025/2026

Module:J2EE, Java EE (JEE) ou Jakarta EE

Compte Rendu :

TP N° 2 –Manipulation de base des Servlets et JSP

Réalisé par:
Taouni Ouiam

Supervisé par:
Pr. Mohamed CHERRADI

Objectif du laboratoire

L'objectif de ce TPest d'apprendre à communiquer les données saisies dans un formulaire entre différents fichiers JSP à l'aide d'un servlet. Plus précisément, il s'agit de comprendre comment :

- Récupérer les informations saisies par l'utilisateur dans un formulaire HTML.
- Transmettre ces informations à un servlet, qui agit comme intermédiaire entre le formulaire et la page de résultat.
- Rediriger les données vers une autre page JSP pour les afficher de manière structurée et sécurisée.
- Séparer clairement la logique métier (traitée par le servlet) et la présentation (gérée par la JSP), ce qui correspond aux bonnes pratiques de développement en Java EE.

Ce TP permet donc de se familiariser avec la communication entre JSP et servlets, ainsi qu'avec la gestion des attributs de requête pour afficher dynamiquement les informations saisies par l'utilisateur.

Table des matières

1-Matériel et technologies utilisés
2-Architecture du projet
3-Flux utilisateur et Implémentation :

1-Page d'accueil (Homepage) :
2-Formulaire d'ajout d'un client
3-Servlet CreerClient
4-JSP protégée (WEB-INF/infoClient.jsp)
5-Formulaire de réservation (JSP)

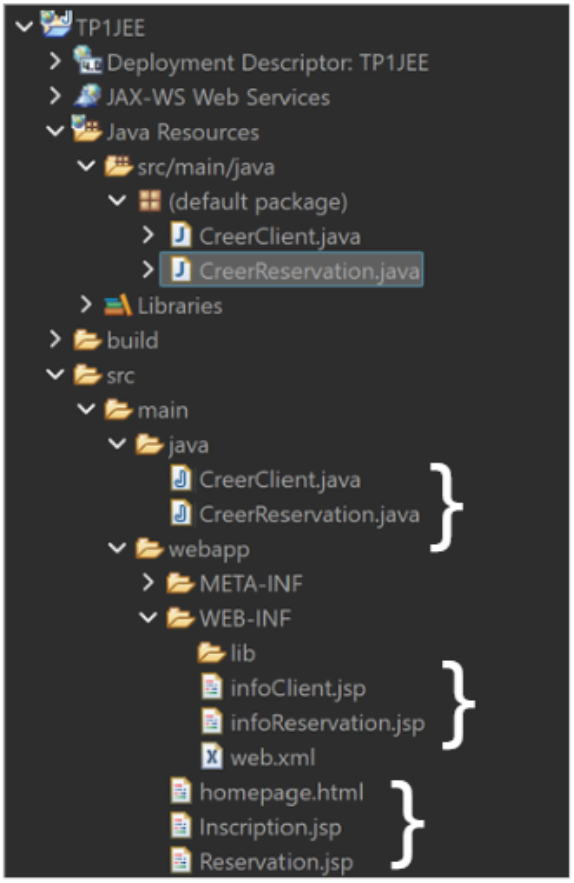
4-Conclusion

1-Matériel et technologies utilisés

- **Java EE** (Servlets, JSP)
- **HttpSession** pour la gestion des sessions
- **JSP** pour les pages web (login.jsp, signup.jsp, welcome.jsp)
- IDE : Eclipse / IntelliJ IDEA
- Serveur : Tomcat

2-Architecture du projet

Ce projet est un **Dynamic Web Project Java EE**, combinant JSP, servlets et filtres pour créer un système d'authentification sécurisé.



Nous avons deux servlets qui servent d'intermédiaires entre les formulaires et les pages de résultat.

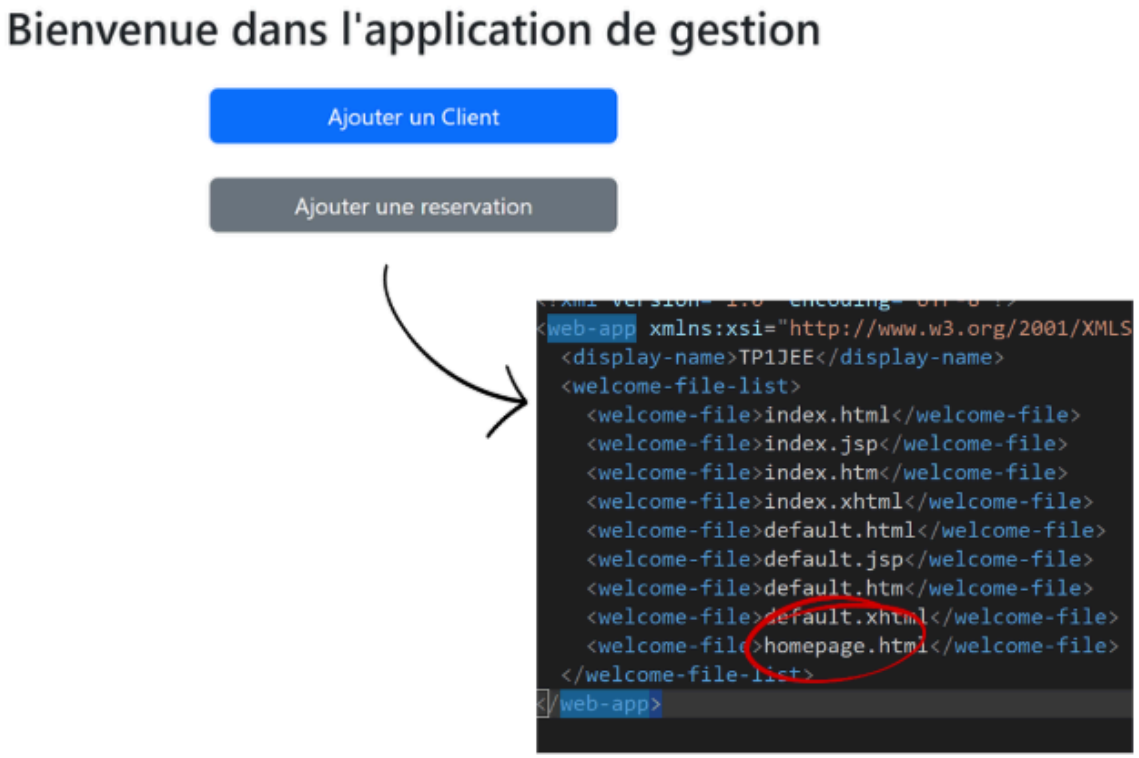
Les deux fichiers JSP placés dans le dossier WEB-INF sont des pages de résultat ou d'affichage final et Ils ne peuvent pas être accédés directement via l'URL

Ils contiennent un fichier HTML et la mise en forme, permettant à l'utilisateur de saisir ses informations.
Ils sont directement accessibles via l'URL

3-Flux utilisateur et Implémentation :

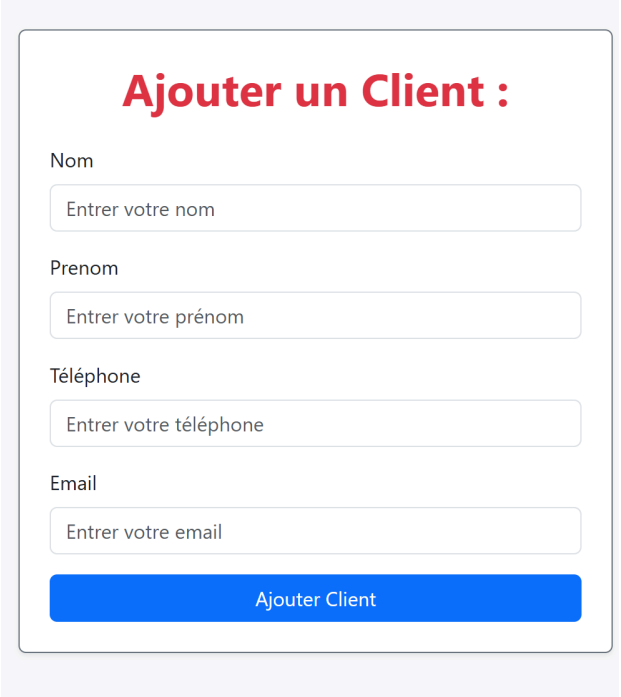
1-Page d’accueil (Homepage) :

La première page que l’utilisateur voit est la homepage, un fichier HTML.



2-Formulaire d'ajout d'un client

Lorsque l'utilisateur clique sur **Ajouter un client**, il est dirigé vers un formulaire contenu dans une page JSP.
Cette JSP contient les champs pour saisir les informations du client : nom, prénom, téléphone et email.
Le formulaire utilise **la méthode POST** et l'action est définie vers le servlet CreerClient.



3-Servlet CreerClient

- Le servlet reçoit les données du formulaire et joue le rôle d'**intermédiaire** entre la JSP du formulaire et la JSP de résultat.
- Il récupère les paramètres avec request.getParameter(...) et les stocke dans des **attributs de requête** (request.setAttribute(...)).
- Ensuite, le servlet **redirige** la requête vers une JSP protégée située dans **WEB-INF** pour afficher les informations du client.

```java
protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

    String nom = request.getParameter("nom");
    String prenom = request.getParameter("prenom");
    String telephone = request.getParameter("telephone");
    String email = request.getParameter("email");


    request.setAttribute("nom", nom);
    request.setAttribute("prenom", prenom);
    request.setAttribute("telephone", telephone);
    request.setAttribute("email", email);


    request.getRequestDispatcher("/WEB-INF/infoClient.jsp").forward(request, response);

}
```

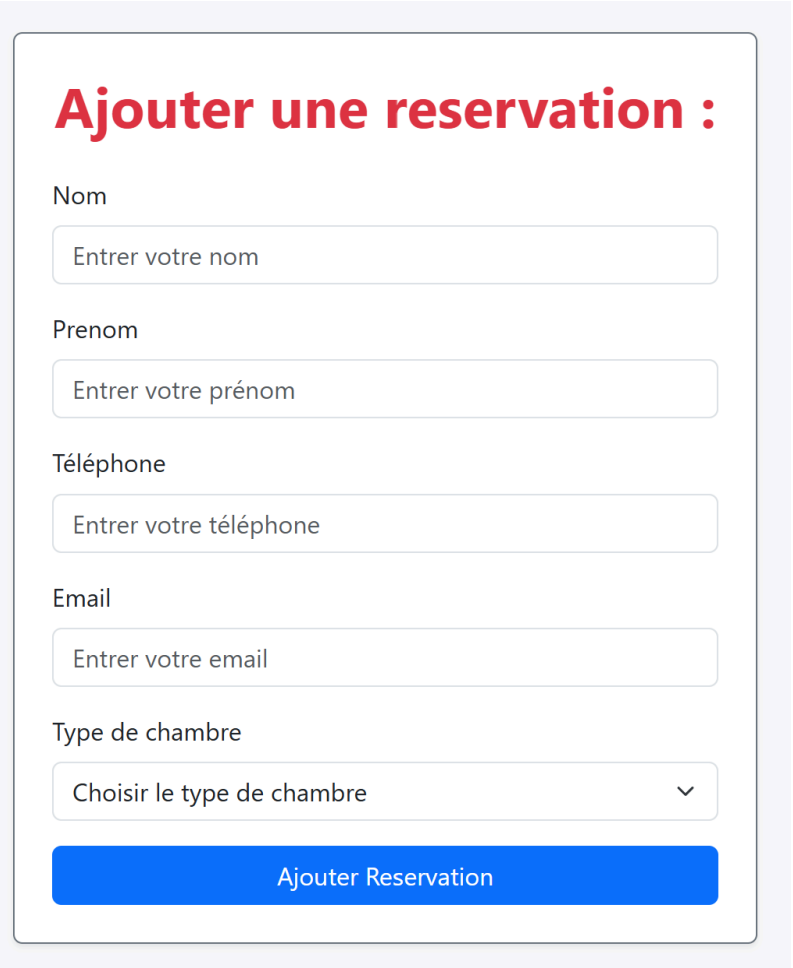
4-JSP protégée (WEB-INF/infoClient.jsp)

- Cette JSP récupère les informations passées par le servlet et les affiche dans un **tableau HTML**.
- Étant placée dans le dossier **WEB-INF**, elle **ne peut pas être accédée directement via l'URL**, garantissant que l'utilisateur passe toujours par le servlet pour voir les données.
- Cela permet de **protéger les données** et de s'assurer que toute la logique métier est exécutée avant l'affichage.

```jsp
                <tr>
                    <td>${nom}</td>
                    <td>${prenom}</td>
                    <td>${telephone}</td>
                    <td>${email}</td>
                </tr>
            </tbody>
        </table>
        <a href="Inscription.jsp" class="btn btn-primary mb-4 d-block w-50 mx-auto">R
    </div>
```

5-Formulaire de réservation (JSP)

En cliquant sur l'un de ces boutons, l'utilisateur est redirigé vers le **formulaire JSP** correspondant pour saisir les informations nécessaires. Une fois le formulaire soumis, un **servlet** récupère les données, les traite et les transmet à une **JSP protégée située dans WEB-INF**, qui affiche les informations sous un format clair et sécurisé. Cette organisation permet de séparer la **logique métier** de la **présentation** et d'assurer que les pages sensibles ne sont pas accessibles directement via l'URL.



Conclusion

Ce laboratoire m'a permis de comprendre comment organiser une application Java EE en séparant la **logique métier**, gérée par les servlets, de la **présentation**, gérée par les JSP. J'ai appris à récupérer les données des formulaires, à les transmettre via des servlets vers des pages protégées dans **WEB-INF**, et à afficher ces informations de manière sécurisée et structurée. Cette expérience m'a permis de maîtriser le **flux utilisateur**, de sécuriser les pages sensibles et de mettre en pratique les bonnes pratiques de développement web en Java EE.